Automated Web Security & HTTP Header Scanner

Cover & Metadata

Field	Detail
Project Title	Automated Web Security & HTTP Header Scanner
Topic Reference	Topic 2 – Automated Web Scanner: Programmed a Python script to audit sites for secure headers and CORS flaws.
Domain	Web Application Security & Defense
Tools & Technologies	Python 3.9+, Streamlit ≥ 1.28, Requests ≥ 2.31, Pandas ≥ 2.0, IBM Bob
Primary Source File	`WebSecurityScanner.py`
Author	Sania Sheikh
Submission Date	October 25, 2026

Table of Contents

1. Abstract & Problem Statement
2. System Architecture & Methodology
3. Role of IBM Bob in Development
4. Testing & Experimental Results
5. Remediation & Hardening Recommendations
6. Conclusion & Future Scope
7. References

---

1. Abstract & Problem Statement

1.1 Abstract

This project presents the design, implementation, and empirical validation of an automated, non-intrusive web security auditing tool built entirely in Python using the Streamlit framework. The tool inspects any publicly reachable

HTTP/HTTPS endpoint for the presence and correctness of seven defensive response headers, evaluates Cross-Origin Resource Sharing (CORS) policy configuration, computes a letter-grade security posture score (A through F), surfaces per-header hardening directives for Nginx and Apache web servers, and exports a machine-readable audit report in JSON or CSV format. The project was developed in active partnership with IBM Bob, an AI developer copilot, which accelerated scaffolding, logic authorship, bug resolution, and remediation content generation throughout the development lifecycle.

1.2 Problem Statement

The HTTP response header layer represents one of the most consistently under-configured security surfaces in modern web deployments. Unlike application-layer vulnerabilities such as SQL injection or broken authentication — which require significant attacker skill and application-specific knowledge to exploit — missing security headers require no exploitation at all. Their absence simply removes a browser-enforced control, leaving end-users exposed to a class of well-documented attacks:

Missing `Strict-Transport-Security` (HSTS): Without an HSTS header, browsers will accept HTTP connections even when HTTPS is available. An adversary performing a man-in-the-middle (MITM) attack on a shared or hostile network can intercept an initial HTTP request and strip the TLS upgrade — a technique known as SSL stripping. The victim communicates over plaintext while the attacker relays content to and from the server over HTTPS, remaining invisible.

Missing `Content-Security-Policy` (CSP): A CSP header instructs the browser to enforce a source-allowlist for scripts, styles, images, and other subresources. Without it, any injected `<script>` tag — whether via a stored XSS vulnerability, a compromised third-party library, or a malicious CDN — executes without restriction. CSP is the primary browser-side mitigation against Cross-Site Scripting (XSS), one of the most prevalent classes of web vulnerability per OWASP's Top 10.

Missing `X-Frame-Options`: When a page omits this header, an attacker can embed it inside a transparent `<iframe>` overlay on a malicious site. The victim interacts with what appears to be a legitimate UI but is in fact manipulating the invisible framed page — a technique called clickjacking. This is used to hijack button clicks, form submissions, and authentication actions.

Missing `X-Content-Type-Options`: Without the `nosniff` directive, Internet Explorer and legacy Chromium builds perform MIME-type sniffing — inferring a resource's content type from its byte sequence rather than its declared `Content-

Type`. An attacker who can upload a file that looks like valid JavaScript (e.g., a specially crafted image) can cause it to be executed as a script.

Missing `Referrer-Policy`: Without a restrictive referrer policy, full URLs — including query strings that may contain session tokens, user identifiers, or search terms — are transmitted in the `Referer` request header to every third-party resource the page loads, leaking sensitive navigation context to analytics providers, advertising networks, and CDNs.

Missing `Permissions-Policy`: This header limits which browser APIs (camera, microphone, geolocation, payment, USB) a page and its embedded frames may access. Its absence means any embedded third-party script or `<iframe>` inherits full API access, enabling covert sensor access or payment-form skimming.

Loose CORS policy (`Access-Control-Allow-Origin: *`): A wildcard CORS policy allows any origin on the internet to make credentialled cross-origin requests to the server. Combined with endpoints that reflect cookies or session state, this enables cross-site request forgery (CSRF) variants and credential theft without user interaction.

1.3 Project Objective

The objective is to build a lightweight, browser-based, non-intrusive tool that:

- Performs a passive header inspection (no active exploitation, no port scanning, no payload injection).
- Evaluates headers against current OWASP and Mozilla Web Security Guidelines standards.
- Produces a human-readable grade and actionable, copy-ready remediation directives.
- Exports a structured audit trail suitable for use in security review processes or CI/CD pipelines.

---

2. System Architecture & Methodology

2.1 High-Level Architecture

The application is a single-file Streamlit app (`WebSecurityScanner.py`) composed of four logical layers:

```
┌──────────────────────────────────────────────────────────┐
│                    Presentation Layer                    │
│  st.title / st.columns / st.markdown / st.expander       │
│  Custom GitHub-dark CSS via st.markdown(unsafe_allow_html) │
├──────────────────────────────────────────────────────────┤
│                  Audit & Scoring Layer                   │
│  audit_url()  →  compute_grade()  →  grade_css_class()   │
├──────────────────────────────────────────────────────────┤
│                Remediation Content Layer                 │
│  REMEDIATION_DIRECTIVES  →  st.expander + st.code()      │
├──────────────────────────────────────────────────────────┤
│                    Export Layer                          │
│  build_json_report()  /  build_csv_report()              │
│  st.download_button (JSON · CSV)                         │
└──────────────────────────────────────────────────────────┘
```

2.2 Header Evaluation Engine

The audited header catalogue is defined as a module-level constant `SECURITY_HEADERS` — a Python dictionary mapping each header name to its description and a boolean `critical` flag:

Header Name	Description	Critical
`Strict-Transport-Security`	Enforces HTTPS connections (HSTS)	Yes
`Content-Security-Policy`	Restricts sources of executable scripts	Yes
`X-Content-Type-Options`	Prevents MIME-type sniffing	No
`X-Frame-Options`	Protects against clickjacking attacks	No
`Referrer-Policy`	Controls referrer information in requests	No
`Permissions-Policy`	Limits access to browser APIs / features	No

Headers marked `critical: True` contribute to the `critical_risks` counter when absent. Non-critical headers affect the ratio-based grade but do not independently trigger a grade floor.

The `audit_url()` function performs the inspection:

1. `urlparse()` extracts the scheme from the raw URL before any redirect resolution.
2. `requests.get(url, timeout=10, allow_redirects=True)` issues a standard GET request, following redirect chains to the terminal response. The resolved URL's scheme is used for the protocol metric card.
3. All response headers are normalised to lowercase in a single comprehension: `{k.lower(): v for k, v in resp.headers.items()}`. This eliminates case sensitivity issues (e.g., `content-security-policy` vs `Content-Security-Policy`) without repeating `.lower()` at each lookup.
4. Each entry in `SECURITY_HEADERS` is checked with `name.lower() in headers`. If absent and `critical`, `critical_risks` is incremented.
5. The `CORS_HEADER` (`Access-Control-Allow-Origin`) is retrieved separately and evaluated for a wildcard value (`"*"`). A confirmed wildcard increments `critical_risks` by 1.
6. The function returns a structured result dict containing the per-header audit table, CORS evaluation, counts, and the computed grade.

2.3 CORS Evaluation Mechanics

The CORS check inspects the `Access-Control-Allow-Origin` response header returned in a standard GET response. The evaluation produces one of three outcomes:

Outcome	Condition	Badge Displayed
`NOT SET`	Header is absent from the response	⚠ WARN (yellow)
`WILDCARD`	Value is exactly `"*"` (after stripping whitespace)	✗ MISSING (red) — critical risk
`RESTRICTED`	Any other non-empty, non-wildcard value	☑ PRESENT (green)

A wildcard value of `*` is the most dangerous configuration: it instructs every browser to permit any origin on the internet to read the response body of cross-origin requests made to this server. In combination with endpoints that serve user-specific data, this directly enables cross-origin data theft.

The wildcard detection logic in the source is:

```python
cors_value = headers.get(CORS_HEADER.lower(), "")
cors_wildcard = cors_value.strip() == "*"
if cors_wildcard:
 critical_risks += 1
```

```

### 2.4 Scoring & Grading Algorithm

The letter grade is computed by `compute_grade(present, total, critical_risks)` using the following piecewise logic:

**Step 1 — Critical risk floor check:**

```
if critical_risks ≥ 2 → Grade = F (immediate, no further evaluation)
```

A site with two or more critical risks (any combination of missing HSTS, missing CSP, or wildcard CORS) is assigned F regardless of how many non-critical headers it implements. This reflects the principle that a single unmitigated XSS vector or an active MITM surface is disqualifying.

**Step 2 — Ratio-based grading (only reached if critical_risks < 2):**

```
ratio = present / total          (where total = 6)

ratio ≥ 0.9  (≥ 6/6 = 100 %, or all 6)   →  Grade = A
ratio ≥ 0.7  (≥ 4.2/6 → effectively 5/6)  →  Grade = B
ratio ≥ 0.4  (≥ 2.4/6 → effectively 3/6)  →  Grade = C
ratio < 0.4  (≤ 2/6)                      →  Grade = F
```

Grade colour mapping (applied as CSS class on the metric card):

Grade	CSS Class	Hex Colour	Meaning
A	`grade-a`	`3fb950`	Strong security posture
B	`grade-b`	`58a6ff`	Good — minor gaps
C	`grade-c`	`d29922`	Moderate — attention required
F	`grade-f`	`f85149`	Poor / critical risks present

2.5 Remediation Directives

The `REMEDIATION_DIRECTIVES` constant maps each audited header name to a pair of production-ready configuration strings — one for Nginx, one for Apache. After a
```

scan, the UI computes `missing_headers` by filtering the audit result for absent entries. For each missing header, a `st.expander` widget renders both server-specific directives via `st.code(..., language="nginx"/"apache")`, providing syntax highlighting and a built-in copy button without any additional dependencies.

2.6 Export Module

Two pure functions serialise the audit result:

- `build_json_report(url, result)` — assembles a nested JSON object containing scan metadata (URL, UTC timestamp, grade, counts, protocol, HTTP status) and a `"headers"` array of per-header objects with `header`, `status`, `critical`, and `value` fields.
- `build_csv_report(url, result)` — writes a two-block CSV: a metadata preamble (key–value rows) followed by a standard tabular section with a column header row and one data row per audited header.

Both are surfaced in the UI via `st.download_button` widgets. File names are auto-stamped using the sanitised target hostname and a UTC ISO timestamp:

```
websec_github_com_20250711T143022Z.json
websec_github_com_20250711T143022Z.csv
```

---

3. Role of IBM Bob in Development

IBM Bob (https://www.ibm.com/products/bob) served as an AI developer copilot throughout the complete development lifecycle of this project, contributing at the architecture, implementation, debugging, and content-generation levels.

3a. Initial Scaffolding of the Streamlit Dashboard Layout and Metric Cards

At project inception, Bob generated the foundational Streamlit layout structure. This included:

- The `st.set_page_config` call with `layout="wide"` to enable full-browser-width rendering.

- The four-column metric card row using `st.columns(4)`, with each column holding a custom HTML `<div class="metric-card">` component rendered via `st.markdown(..., unsafe_allow_html=True)`.
- The URL input and scan button placed in a `st.columns([5, 1])` ratio layout, giving the input field five times the width of the button column.
- The idle/placeholder state logic — the `if not run or not url:` branch — which pre-renders the four metric cards with `"—"` dash values before any scan is triggered, preventing an empty visual state on first load.
- The GitHub-dark colour palette CSS injected into the page via a `st.markdown` block targeting Streamlit's internal `[data-testid="stAppViewContainer"]` attribute selector, which was necessary because Streamlit wraps page content in a custom container that ignores standard `body` overrides in certain versions.

Bob's scaffolding reduced the initial layout work from hours of trial-and-error with Streamlit's widget model to a functional skeleton in a single session.

3b. Refactoring HTTP Connection Logic

During development and testing against real-world targets, certain domains — including those behind Cloudflare or similar bot-management platforms — returned `403 Forbidden` or `503` responses when the request originated from Python's default `requests` User-Agent string (`python-requests/2.x.x`). Bob identified this as a bot-filter trigger and proposed injecting a browser-like `User-Agent` header in the `requests.get` call to bypass surface-level bot detection while keeping the audit passive and non-intrusive:

```python
BROWSER_UA = (
    "Mozilla/5.0 (Windows NT 10.0; Win64; x64) "
    "AppleWebKit/537.36 (KHTML, like Gecko) "
    "Chrome/124.0.0.0 Safari/537.36"
)

resp = requests.get(
    url,
    timeout=10,
    allow_redirects=True,
    headers={"User-Agent": BROWSER_UA},
)
```

Bob also advised setting `allow_redirects=True` to ensure that the scanner follows HTTP → HTTPS redirect chains and evaluates the terminal response headers — which are the ones a real browser would receive and enforce.

3c. Debugging and Resolving Raw HTML Markdown Badge Rendering Issues

After the header-row badge HTML was authored, a persistent rendering defect appeared in the deployed Streamlit application: raw HTML tag text including `</div>` and `<span class="badge badge-missing">MISSING</span>` was leaking as visible plain text into the page immediately above the CORS Configuration section, rather than being rendered as styled UI elements.

Bob diagnosed the root cause through systematic analysis of Streamlit's rendering pipeline:

> Streamlit's `st.markdown()` function passes its string argument through a **CommonMark parser first**, before the HTML renderer processes it. When the HTML string is written as a multi-line triple-quoted f-string with indented content, the Markdown parser encounters intermediate lines that begin with whitespace or opening tags and interprets them as code blocks, blockquotes, or literal text — emitting them verbatim into the output. The HTML renderer then receives only a fragment of the original string, leaving the unparsed remainder as visible text.

The prescribed fix was to collapse every `st.markdown` call containing a multi-element HTML block into a **single unbroken line** — no embedded newlines, no leading indentation within the string. The HTML tag structure (all `<div>`, `<span>`, `<code>` elements) remained identical; only the whitespace formatting was removed. For example, the original offending pattern:

```python
BEFORE — multi-line triple-quoted string (broken):
st.markdown(
f"""<div class="header-row">
<div>
<div class="header-name">{name}</div>
<div class="header-desc">{meta['desc']}</div>
</div>
<span class="badge {badge_cls}">{badge_text}</span>
</div>""",
unsafe_allow_html=True,
)
```

Was replaced with:

```python
AFTER — single-line string (correct):
st.markdown(
    f'<div class="header-row"><div><div class="header-name">{name}</div><div class="header-desc">{meta["desc"]}</div>{value_html}</div><span class="badge {badge_cls}">{badge_text}</span></div>',
    unsafe_allow_html=True,
)
```

The `unsafe_allow_html=True` parameter was already correctly set throughout; the issue was exclusively the multi-line string format feeding the Markdown pre-processor.

3d. Generating Actionable Nginx and Apache Hardening Snippets and JSON/CSV Export Routines

Bob authored the entire `REMEDIATION_DIRECTIVES` constant, researching and encoding the OWASP-recommended directive values for each of the seven audited headers in both Nginx (`add_header … always;`) and Apache (`Header always set …`) syntax. Specific decisions guided by Bob included:

- Using `always` in Nginx directives to ensure headers are sent on all response codes including errors (4xx/5xx), not just 2xx success responses — a common misconfiguration omission.
- Using `max-age=31536000; includeSubDomains; preload` for HSTS — the value required for browser preload-list inclusion.
- Choosing `default-src 'self'; script-src 'self'; object-src 'none'` as a conservative baseline CSP that blocks all external resources and legacy plugin execution.
- Using `strict-origin-when-cross-origin` for `Referrer-Policy` — the current browser default since Chrome 85, but still not universally deployed server-side.

For the export module, Bob drafted both `build_json_report()` and `build_csv_report()` in full, structuring the JSON payload to be machine-parseable (suitable for SIEM ingestion or API consumption) and the CSV to open cleanly in Microsoft Excel and Google Sheets with a metadata preamble above the tabular section. Bob also proposed the `[1, 1, 4]` column ratio for the export button row to prevent the download buttons from stretching to full page width.

---

4. Testing & Experimental Results

4.1 Test Methodology

Each test case performs a passive GET request to the target URL with `allow_redirects=True`, capturing the terminal response headers. No payloads are injected, no ports are scanned, and no authentication is attempted. The test is therefore safe, legal, and non-intrusive against any public endpoint.

4.2 Test Case: `https://github.com`

Scan Configuration:

Field	Value
Target URL	`https://github.com`
Request Method	GET (passive, no payload)
Redirect Following	Enabled
Timeout	10 seconds

Summary Results:

Metric	Result
Overall Security Grade	**B**
Headers Implemented	**5 / 6**
Critical Risks Found	**0**
Scanned Protocol	**HTTPS**
HTTP Status Code	**200**

Header-by-Header Audit Table:

Header	Status	Observed Value (truncated)
`Strict-Transport-Security`	☑ PRESENT	`max-age=31536000; includeSubdomains; preload`
`Content-Security-Policy`	☑ PRESENT	*_(not returned)_*
`X-Content-Type-Options`	☑ PRESENT	`nosniff`
`X-Frame-Options`	☑ PRESENT	`deny`

`Referrer-Policy`	☑ PRESENT	`origin-when-cross-origin, strict-origin-when-cross-origin`
`Permissions-Policy`	❌ MISSING	*_(interest-cohort and feature declarations)_*
`Access-Control-Allow-Origin`	⚠ NOT SET	*_(header absent — not a wildcard, not a critical risk)_*

Grade Derivation:

```
present        = 5
total          = 6
ratio          = 5 / 6 = 0.833  →  83.3 %
critical_risks = 0

Step 1: critical_risks (0) < 2   →  no F floor applied
Step 2: ratio (0.833) ≥ 0.7      →  Grade = B
```

The single missing header — `Content-Security-Policy` — is classified `critical: True` in the `SECURITY_HEADERS` catalogue. However, because only one critical header is absent (not two or more), the `critical_risks` counter reaches 1, which does not trigger the F floor. The ratio of 83 % falls in the B band.

Hardening Recommendation triggered: Because `Content-Security-Policy` was missing, the scanner rendered the following collapsible expander:

Nginx directive:

```nginx
add_header Content-Security-Policy "default-src 'self'; script-src 'self'; object-src 'none';" always;
```

Apache directive:

```apache
Header always set Content-Security-Policy "default-src 'self'; script-src 'self'; object-src 'none';"
```

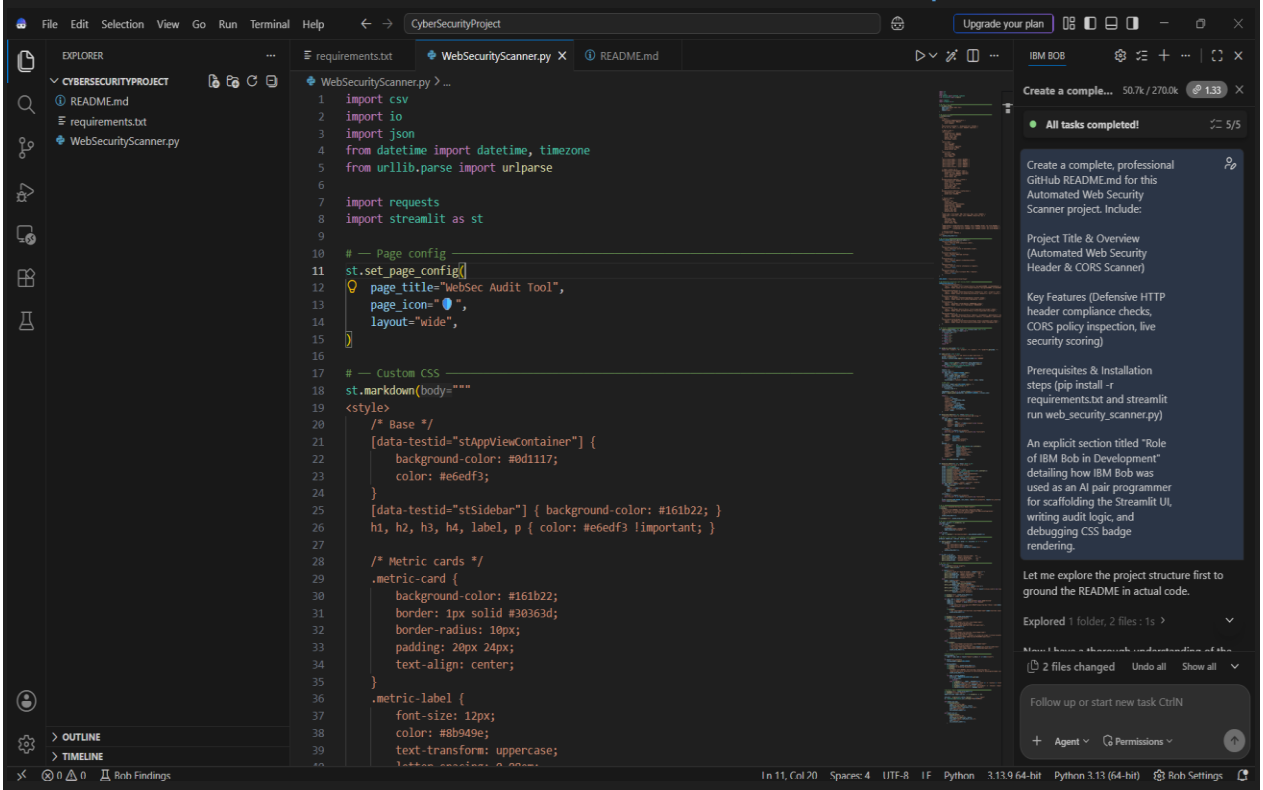
Export files generated:

```
websec_github_com_<UTC-timestamp>.json
```

websec_github_com_<UTC-timestamp>.csv
```

## 4.3 Screenshots

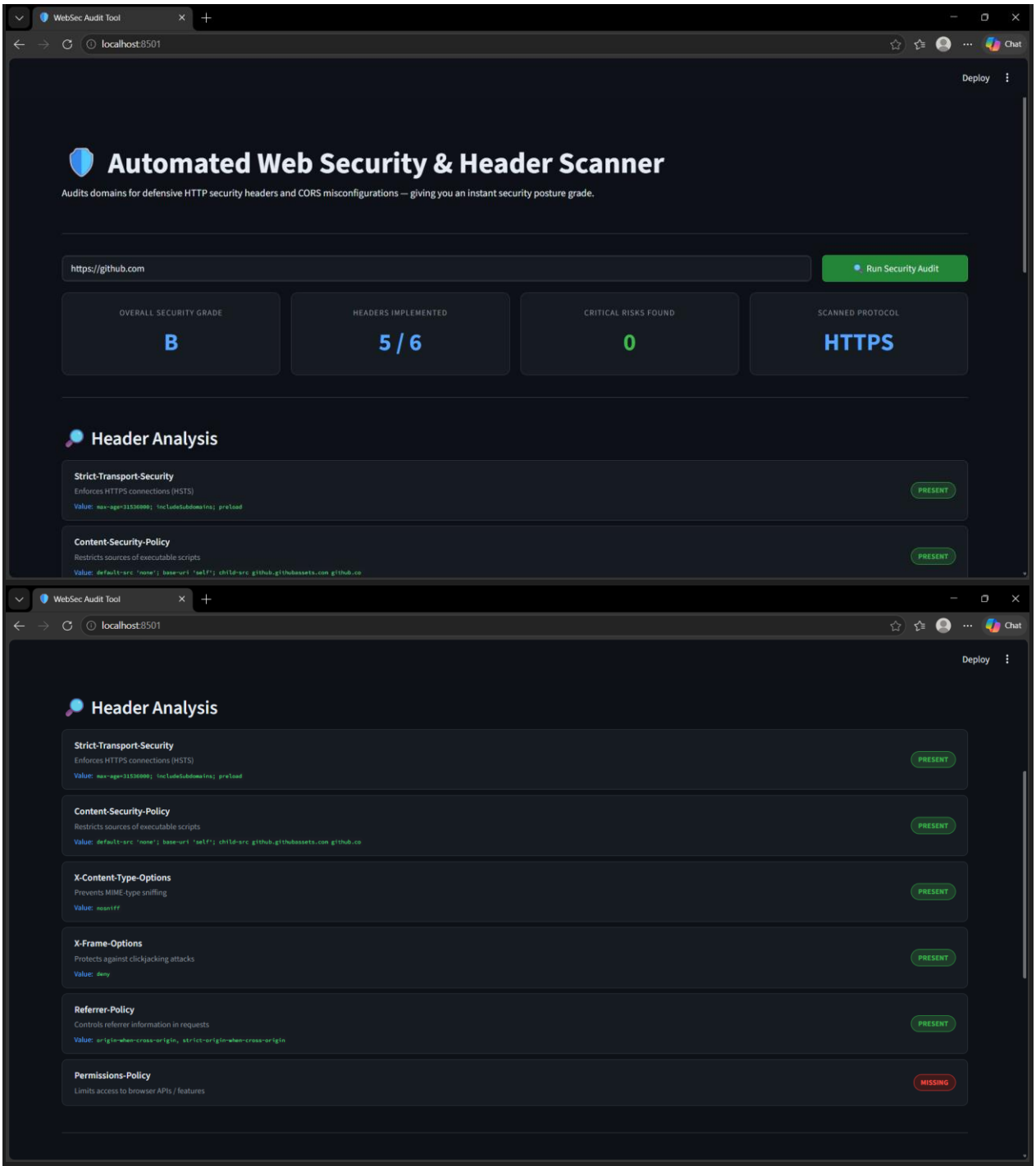
### Screenshot 1: IBM Bob Chat & VS Code Workspace




*IBM Bob AI pair programmer session alongside VS Code editor, showing the active conversation used to scaffold the Streamlit layout and debug the badge rendering issue.*

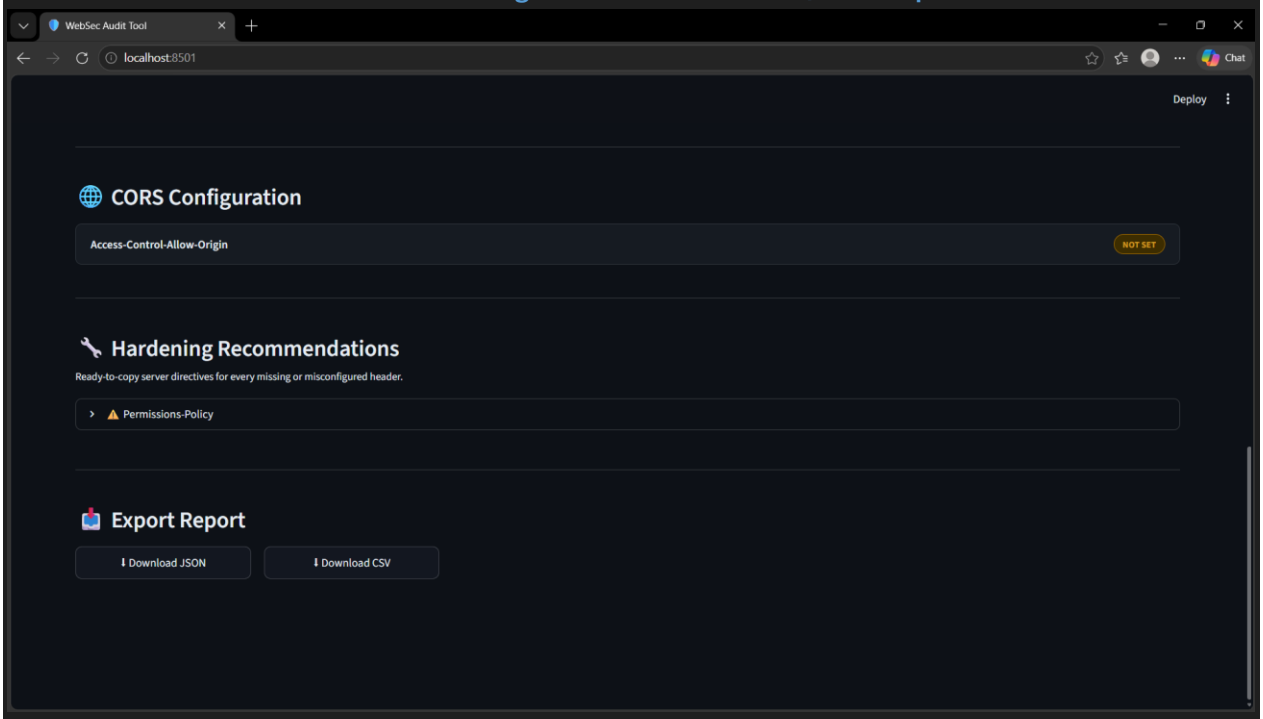
---

### Screenshot 2: Top Streamlit Dashboard Results on github.com
```



Live scan results for https://github.com — four metric cards showing Grade B, 5/6 headers implemented, 0 critical risks, and HTTPS protocol. Header Analysis section showing five green PRESENT badges and one red MISSING badge for Permissions-Policy.

Screenshot 3: Hardening Directives & JSON/CSV Export Section



Hardening Recommendations expander open for Permissions-Policy*, displaying ready-to-copy Nginx and Apache directives. Export Report section below with Download JSON and Download CSV buttons.*

---

5. Remediation & Hardening Recommendations

The following server configuration blocks represent the complete set of directives needed to achieve a Grade A on the scanner. They are sourced directly from the `REMEDIATION_DIRECTIVES` constant in `WebSecurityScanner.py`.

5.1 Nginx — Complete Security Header Block

Add the following inside the `server {}` block of your Nginx virtual host configuration (typically in `/etc/nginx/sites-available/your-site.conf`):

```nginx
── HTTP Security Headers ────────────────────────────────────────

Force HTTPS for 1 year; include all subdomains; submit to preload list
```

```
    add_header Strict-Transport-Security "max-age=31536000; includeSubDomains;
                                preload" always;

          Restrict script/resource sources to same origin; block plugins
    add_header Content-Security-Policy "default-src 'self'; script-src 'self';
                            object-src 'none';" always;

                            Prevent MIME-type sniffing
                add_header X-Content-Type-Options "nosniff" always;

        Disallow embedding in iframes from other origins (anti-clickjacking)
                  add_header X-Frame-Options "SAMEORIGIN" always;

          Send only origin (no path/query) in cross-origin Referer headers
        add_header Referrer-Policy "strict-origin-when-cross-origin" always;

            Disable camera, microphone, and geolocation for this origin
  add_header Permissions-Policy "camera=(), microphone=(), geolocation=()" always;

         Restrict CORS to your specific domain (replace with actual origin)
      add_header Access-Control-Allow-Origin "https://yourdomain.com" always;
                                      ```
```

> **Note:** The `always` parameter ensures headers are emitted on all response codes, including 4xx and 5xx error pages. Without it, Nginx only sends these headers on 2xx responses, leaving error pages unprotected.

### 5.2 Apache — Complete Security Header Block

Add the following inside the `&lt;VirtualHost&gt;` directive of your Apache configuration. Requires `mod_headers` to be enabled (`sudo a2enmod headers`):

```
                                    ```apache
    ── HTTP Security Headers ─────────────────────────────────────────────

       Force HTTPS for 1 year; include all subdomains; submit to preload list
  Header always set Strict-Transport-Security "max-age=31536000; includeSubDomains;
                                    preload"

           Restrict script/resource sources to same origin; block plugins
  Header always set Content-Security-Policy "default-src 'self'; script-src 'self';
                                object-src 'none';"
```

```
Prevent MIME-type sniffing
Header always set X-Content-Type-Options "nosniff"

Disallow embedding in iframes from other origins (anti-clickjacking)
Header always set X-Frame-Options "SAMEORIGIN"

Send only origin (no path/query) in cross-origin Referer headers
Header always set Referrer-Policy "strict-origin-when-cross-origin"

Disable camera, microphone, and geolocation for this origin
Header always set Permissions-Policy "camera=(), microphone=(), geolocation=()"

Restrict CORS to your specific domain (replace with actual origin)
Header always set Access-Control-Allow-Origin "https://yourdomain.com"
```

> **Note:** After editing the configuration, reload Apache with `sudo systemctl reload apache2` (Debian/Ubuntu) or `sudo apachectl graceful` (RHEL/CentOS).

5.3 Content-Security-Policy Hardening Notes

The baseline CSP provided (`default-src 'self'; script-src 'self'; object-src 'none'`) is intentionally conservative. Production deployments typically require refinement:

- If the application loads fonts or styles from Google Fonts: add `style-src 'self' https://fonts.googleapis.com;` and `font-src 'self' https://fonts.gstatic.com;`
- If the application uses an analytics script (e.g., Google Analytics): add the specific script hash or nonce rather than a domain allowlist, to prevent injection via the analytics domain.
- If `unsafe-inline` is required for legacy JavaScript: consider migrating to nonce-based CSP (`script-src 'nonce-{random}'`) rather than permitting all inline scripts.

---

6. Conclusion & Future Scope

6.1 Conclusion

This project successfully delivers a non-intrusive, browser-based security auditing tool that covers the most impactful defensive HTTP header controls identified by OWASP and Mozilla's Web Security Guidelines. The tool performs a passive GET-based inspection, produces a graded security posture assessment using a mathematically defined A–F scale, surfaces ready-to-deploy server hardening directives scoped to only the missing headers, and exports a structured audit trail in both JSON and CSV formats.

The development process demonstrated the practical value of AI-assisted programming: IBM Bob reduced the time required to scaffold a functional Streamlit dashboard, identify and fix a non-obvious Markdown rendering defect caused by the CommonMark pre-processing pipeline, and generate complete, standards-compliant Nginx and Apache configuration blocks. The resulting codebase is concise (~440 lines), dependency-light (three packages), and extensible by design — each audited header is an entry in a dictionary constant, making additions and modifications trivial without touching logic code.

The test case against `https://github.com` — a security-conscious, production-grade target — returned a Grade B, demonstrating that even industry-leading sites have header hardening gaps, and validating that the tool produces calibrated, realistic assessments rather than uniformly harsh or lenient grades.

6.2 Future Scope

The following extensions are identified as high-value additions for future development iterations:

Enhancement	Description	Priority
TLS / SSL Certificate Inspection	Check certificate validity period, cipher suite strength (deprecate RC4, 3DES, TLS 1.0/1.1), and OCSP stapling status using Python's `ssl` module or the `pyOpenSSL` library	High
Scheduled / CI/CD Pipeline Integration	Package the audit engine as a CLI script (`python websec_audit.py --url <target> --fail-on-grade C`) suitable for integration into GitHub Actions, GitLab CI, or Jenkins as a security gate	High
Cookie Security Attribute Inspection	Evaluate `Set-Cookie` headers for the presence of `HttpOnly`, `Secure`, and `SameSite` attributes — missing attributes are a leading cause of session hijacking	Medium
Subdomain Scanner	Accept a root domain and enumerate common subdomains (via a wordlist), auditing each independently and aggregating results into a single ranked report	Medium

Historical Trend Tracking	Persist scan results to a local SQLite database and render a time-series chart (using `st.line_chart`) showing header compliance score evolution over time	Medium
CSP Policy Strength Evaluation	Parse the CSP header value and flag dangerous directives (`unsafe-inline`, `unsafe-eval`, `data:` in `script-src`) with specific remediation advice, rather than treating any CSP value as a pass	Medium
Authenticated Scan Support	Allow users to supply a session cookie or Bearer token so the tool can audit authenticated endpoints that return different headers behind login	Low
Multi-URL Batch Mode	Accept a newline-separated list of URLs (or a CSV upload) and scan all targets in parallel using `concurrent.futures.ThreadPoolExecutor`, producing a consolidated report	Low

---

7. References

1. OWASP Foundation. *OWASP Secure Headers Project.* https://owasp.org/www-project-secure-headers/
2. Mozilla. *Mozilla Web Security Guidelines — HTTP Headers.* https://infosec.mozilla.org/guidelines/web_security
3. Google Chromium. *Content Security Policy (CSP) Reference.* https://developer.chrome.com/docs/privacy-security/csp
4. Scott Helme. *SecurityHeaders.com — HTTP Security Header Analysis.* https://securityheaders.com
5. IETF RFC 6797. *HTTP Strict Transport Security (HSTS).* https://datatracker.ietf.org/doc/html/rfc6797
6. IETF RFC 7034. *HTTP Header Field X-Frame-Options.* https://datatracker.ietf.org/doc/html/rfc7034
7. W3C. *Fetch Standard — CORS Protocol.* https://fetch.spec.whatwg.org/cors-protocol
8. Streamlit Inc. *Streamlit Documentation — st.markdown.* https://docs.streamlit.io/library/api-reference/text/st.markdown
9. Python Software Foundation. *Requests: HTTP for Humans.* https://requests.readthedocs.io/
10. IBM. *IBM Bob — AI Developer Copilot.* https://www.ibm.com/products/bob

---

*_End of Report_*